



AI Doorbell

Project Engineering

Year 4

Eoin Fitzpatrick

Bachelor of Engineering (Honours) in Software and

Electronic Engineering

Atlantic Technological University

2025/2026

FYP

Live

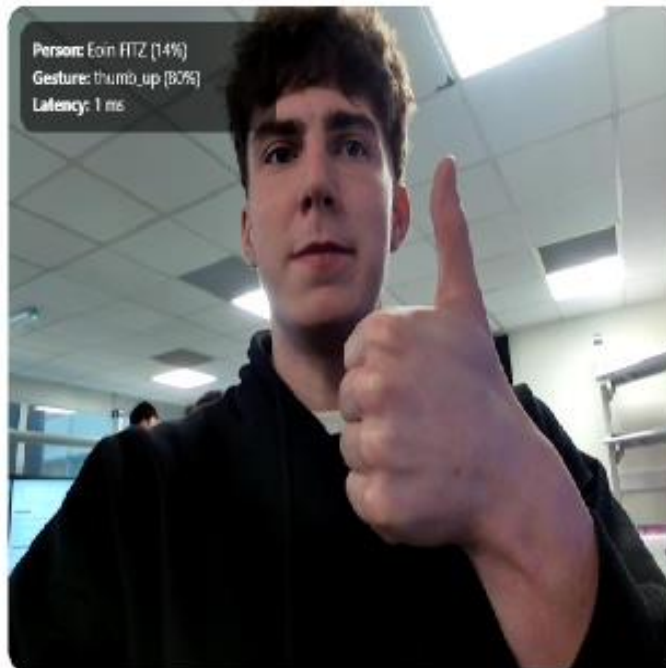
Enroll

Simulation

Results

Simulation

Connected



Controls

Stop

Seconds left: 3

Door State

DOOR OPENING...

Detected gesture: thumb_up
thumbs_up = Open
open_palm = Close

Current Result

Person: Eoin FITZ
Face conf: 0.142
Gesture: thumb_up
Gesture conf: 0.8
Latency: 1 ms
Distance: 0.4292

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Eoin Fitzpatrick

Acknowledgements

I would like to sincerely thank my supervisor, Brian, for his guidance, support, and feedback throughout this project. His advice and direction helped me during the different stages of planning, and development, and helped me stay focused on the main goals of the project.

I would also like to thank the lecturers in the Department of Software and Electronic Engineering for the knowledge and skills I developed over the course of my degree, which helped me throughout this project.

Table of Contents

1	Summary	8
2	Poster	9
3	Introduction	10
4	Background	11
4.1	Alzheimer's Disease and Memory Difficulties	11
4.2	Assistive Technology for Independent Living	11
4.3	Facial Recognition as an Assistive Tool	12
4.4	Gesture Recognition for Simple Interaction	12
4.5	Real-Time Web-Based Systems.....	13
4.6	Relevance to This Project.....	13
5	Project Architecture	14
6	Project Plan	16
7	Implementation	21
7.1	Face Recognition	21
7.2	Gesture Recognition and Model Training.....	23
7.3	Database Design.....	25
7.4	Backend Development.....	27
7.5	Frontend Development.....	29
7.6	System Integration.....	30
8	Testing and Evaluation.....	32
8.1	Testing Approach	32
8.2	Face Recognition Testing	32
8.3	Gesture Recognition Testing.....	33

8.4	Backend and Service Testing.....	34
8.4.1		36
8.5	Overall Evaluation	36
8.6	Limitations and Future Work	37
9	Ethics.....	39
10	Conclusion	41
11	References	42

1 Summary

This project focused on developing a smart doorbell assistant designed to help older people, particularly those living with Alzheimer's disease or memory loss, recognise who is at their door. For people with memory difficulties, recognising familiar faces can become stressful and confusing, especially in unexpected situations such as answering the door. The main aim of this project was to create a working prototype that could identify known visitors in real time and display that information clearly to the user, helping them feel more confident and secure in their home.

The system was developed as a web-based application using a React frontend and a FastAPI backend. A live webcam feed was used to capture video at the door, while the backend processed incoming image frames and performed facial recognition against stored user data in a database. When a known face was detected, the system returned the visitor's identity in real time. The project also explored gesture recognition using MediaPipe, allowing the user to interact with the system through simple hand gestures. In the prototype, these gestures were used to simulate door control actions, such as opening or locking the door for recognised visitors like family members or carers.

A major part of the project involved integrating the different technologies into one real-time pipeline and improving performance so that recognition results could be displayed smoothly and reliably. Testing was carried out on the main features including facial recognition, enrolment, gesture processing, and backend services. Overall, the final prototype demonstrates how artificial intelligence can be applied in a practical and meaningful way to support assisted living. The system shows clear potential as a tool that could reduce uncertainty and improve independence for older users when responding to visitors at their home

2 Poster



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

AI Smart Doorbell

Eoin Fitzpatrick | B.Eng. (Hons) Software & Electronic Engineering | Final Year Project

React + Vite

FastAPI + WebSocket

SQLite + ML Pipeline

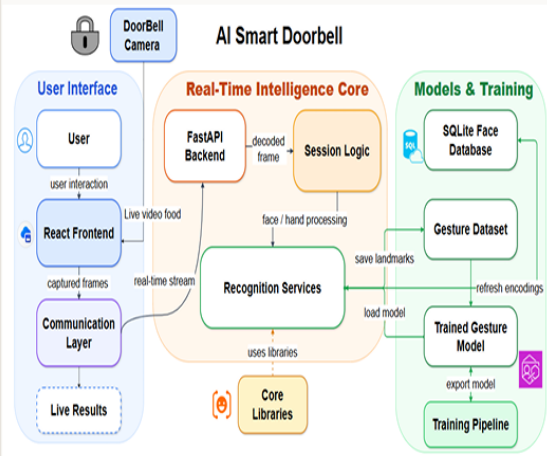
Project Overview

AI Smart Doorbell is an assistive system designed to help older people particularly those living with Alzheimer's or memory loss recognise who is at their door. A live webcam feed is processed in real time to identify known visitors such as family members or carers, displaying their name clearly on screen. Users can also respond using simple hand gestures to simulate opening or locking the door, with all data stored privately on a local device.

Key Deliverables

Real-time face recognition displayed in the browser
Simple gesture control — thumbs up to open, palm to lock
Local enrolment and storage, no cloud or internet required

Architecture Diagram



The diagram illustrates the system architecture. A DoorBell Camera feeds into a User Interface (User, React Frontend, Communication Layer, Live Results). The React Frontend sends a 'Live video feed' to the Real-Time Intelligence Core (FastAPI Backend, Session Logic, Recognition Services, Core Libraries). The Real-Time Intelligence Core sends a 'decoded frame' to Session Logic and a 'face / hand processing' signal to Recognition Services. Recognition Services interacts with Models & Training (SQLite Face Database, Gesture Dataset, Trained Gesture Model, Training Pipeline). Recognition Services sends 'save landmarks' to the Gesture Dataset and 'load model' to the Trained Gesture Model. The Trained Gesture Model sends 'refresh encodings' to the Gesture Dataset and 'export model' to the Training Pipeline. Core Libraries are used by Recognition Services.

User Experience Modes

Enroll
Add known visitors like family or carers stored privately on device
Name: _____
Samples: 10

Simulation
Use a hand gesture to open or lock the door for a recognised visitor
DOOR OPENING

Technical Highlights

All face and visitor data stored locally no cloud or internet required.
Each visitor session is handled independently for reliable, accurate results.
Gesture model trained and validated on 513 samples with 100% test accuracy.

Measured Results from the Repository

100% Gesture accuracy (2 classes)	200 Trees in gesture classifier	64 Hand landmark features	103 Test samples	80ms Local WebSocket latency
--------------------------------------	------------------------------------	------------------------------	---------------------	---------------------------------

Conclusion and Future work

All project objectives were successfully met, delivering a real-time assistive doorbell system proven to support older people with memory difficulties.
Future Work: mobile app with visitor notifications and integration with a real smart lock.

Page 9 of 44

3 Introduction

As people grow older, everyday tasks that once felt simple can become more difficult, especially for those living with Alzheimer's disease or other forms of memory loss. One situation that can cause confusion or stress is answering the door and not being able to recognise who is outside. Forgetting familiar faces can affect confidence, independence, and personal safety in the home. Because of this, there is growing value in assistive technologies that can support older adults in a practical and easy-to-use way.

The aim of this project was to design and develop a smart doorbell assistant that helps older people recognise known visitors using facial recognition. The main idea behind the project was to create a system that captures a live video feed at the door, processes the image in real time, and identifies familiar people for the user. Rather than focusing only on security, the project was designed with an assistive purpose, to support people who may struggle with memory and recognition in day-to-day life. A further aim was to allow simple interaction with the system through hand gestures, giving the user a way to respond to recognised visitors by opening or locking the door.

The system was developed as a web-based prototype using a React frontend and a FastAPI backend, with WebSockets used to support real-time communication between both sides. Facial recognition was used as the main feature for identifying enrolled users, while gesture recognition using MediaPipe was explored to detect hand signals linked to door actions. An enrolment feature was also included so that new users could be added to the recognition database, making the system more flexible and practical for repeated use in a home setting.

The scope of this project was to design, build, integrate, and test a functional prototype rather than a full commercial product. The work focused on combining live video capture, recognition technology, backend processing, gesture-based interaction, and a user-facing interface into one working system. The final outcome is a prototype that demonstrates how AI-based technology can be used in a meaningful way to support assisted living and improve quality of life for older people experiencing memory difficulties.

4 Background

4.1 Alzheimer's Disease and Memory Difficulties

Alzheimer's disease is one of the most common causes of dementia and affects memory, thinking, and the ability to carry out daily activities. It usually develops gradually and becomes more severe over time. While dementia affects people in different ways, memory loss, confusion, and difficulty recognising familiar people are all well-known symptoms. The World Health Organization describes dementia as a condition that affects memory, thinking and the ability to perform everyday activities, while the NHS notes that a person with Alzheimer's disease may increasingly struggle to remember the names of people they know and may even find it difficult to recognise family members or friends.

For older adults living with Alzheimer's or other forms of memory loss, everyday situations can become stressful when they involve uncertainty. One example of this is answering the door. If the person standing outside is not immediately recognised, this can create confusion, anxiety, and hesitation. In a home setting, this is important not only from a practical point of view, but also from a wellbeing and safety point of view. Alzheimer's Society also highlights that dementia can affect communication, perception, and the ability to interpret what is being seen, which helps explain why familiar situations may still become difficult for a person living with the condition.

Because of this, there is a clear need for supportive technologies that can help reduce uncertainty in day-to-day life. In the context of this project, the goal is not to replace a carer or family member, but to provide an assistive tool that helps the user recognise known visitors and respond with more confidence.

4.2 Assistive Technology for Independent Living

Assistive technology is often used to support older adults in maintaining independence, safety, and quality of life within their own homes. In recent years, there has been growing interest in using digital and smart-home technologies to support people living with cognitive decline. Research reviews in this area show that assistive technologies can contribute to everyday living, safety, and independent ageing, especially when they are designed around real user needs in the home environment.

This project fits within that wider assistive technology space. Rather than treating the system as a standard security product, it is more accurate to describe it as an assistive smart doorbell

prototype. Its purpose is to support recognition and decision-making at the door by giving the user more information in real time. This makes the project more meaningful than a general face recognition demo, as it applies AI in a way that directly addresses a real-world challenge faced by some older adults.

An important part of assistive technology design is usability. A system intended for older users should be simple, clear, and easy to interact with. For that reason, the project focuses on a small number of core functions identifying a visitor, displaying their identity if recognised, and allowing simple gesture-based simulated responses. Keeping the system focused in this way improves clarity and makes the prototype easier to understand and evaluate.

4.3 Facial Recognition as an Assistive Tool

Facial recognition is a computer vision technique used to detect and identify faces in images or video. In general, a facial recognition system captures an image, locates the face, extracts facial features, and compares them with previously stored data to determine whether there is a match. In this project, facial recognition is used to identify known visitors such as family members or carers and return that result to the user through the web interface.

The technical value of facial recognition in this project is that it allows the system to work in real time from a live camera feed. The `face_recognition` library, which is built on top of `dlib`, is commonly used in Python applications for face detection and recognition, and its documentation describes it as a simple way to recognise and manipulate faces from Python. Based on that type of workflow, a recognition system can compare faces captured by the camera with enrolled face encodings stored in a database and return either a recognised identity or an unknown result.

In the context of this project, facial recognition is not being used primarily for surveillance or strict access control. Instead, it is being used as an assistive feature to support recognition and reassurance. That distinction is important. The purpose of the system is to help the user understand who is at the door, not to make fully autonomous decisions on their behalf.

4.4 Gesture Recognition for Simple Interaction

Alongside facial recognition, this project also explores gesture recognition as a secondary interaction method. Gesture recognition allows the system to interpret hand movements captured by the camera and map them to application actions. Google's MediaPipe documentation explains that its hand landmark and gesture recognition tools can detect hand keypoints and recognise gestures in real time from image streams.

In this project, gestures are used to simulate door-response actions for recognised visitors. For example, if a known family member or carer is identified, the user can perform a simple hand gesture to trigger a simulated response such as "open" or "lock." As this is a prototype, the system does not control a real physical lock. Instead, the gesture feature is included to show how contactless interaction could be integrated into an assistive doorbell system.

This is useful from both a design and accessibility point of view. Gesture-based input offers a simple alternative to more complex controls and supports the idea of quick interaction in front of a live camera interface. It also makes the prototype feel more complete by showing how recognition can be linked to an action rather than only displaying a result.

4.5 Real-Time Web-Based Systems

To make the prototype practical, the project was developed as a real-time web application. A live camera feed is captured on the frontend and sent to the backend for processing, after which the recognition result is returned and displayed to the user. FastAPI provides support for WebSockets, which allow data to be sent and received continuously between client and webserver rather than through separate one-off HTTP requests. The FastAPI documentation describes WebSockets to read data from the client and send data back through a persistent connection, which makes them suitable for live interactive applications.

This real-time structure is important for the aims of the project. A delayed result would reduce the usefulness of the system in a real doorbell scenario. By combining a web frontend, backend processing, facial recognition, gesture recognition, and a stored enrolment database, the project creates a single pipeline that demonstrates how these technologies can work together in an assistive setting.

4.6 Relevance to This Project

The background areas discussed above directly support the motivation and design of this project. Alzheimer's disease and dementia can affect memory, recognition, and confidence in daily situations. Assistive technology offers a way to support independent living in the home. Facial recognition provides the technical method for identifying known visitors, while gesture recognition provides a simple way to simulate responses once a person has been recognised. Together, these ideas form the basis of the smart doorbell assistant developed in this project.

Overall, the background to this project is not just technical. It is also human-centred. The value of the system comes from applying modern AI tools to a real-world problem in a way that is practical, understandable, and focused on supporting the user.

5 Project Architecture

System architecture of the smart doorbell assistant shows the overall architecture of the smart doorbell assistant developed in this project. The system was designed as a real-time web-based prototype that combines facial recognition, gesture recognition, backend processing, and stored user data into one application. The architecture is divided into three main areas the user interface, the real-time intelligence core, and the data, models, and training components.

On the user side, the React frontend provides the main interface for live recognition, enrolment, and gesture training. It captures webcam frames and sends them through a communication layer using WebSockets and base64 JPEG frames. This allows a continuous stream of image data to be passed from the frontend to the backend for real-time processing.

At the centre of the system is the FastAPI backend, which manages application startup, routing, and real-time communication. Session logic is used to coordinate face persistence, gesture smoothing, and latency measurement. The backend then passes incoming frame data to the recognition services, which include face recognition, gesture recognition, enrolment handling, and frame processing. These services rely on core libraries such as OpenCV, the Python face recognition library, and MediaPipe.

The right-hand side of the architecture contains the data and training components. A SQLite database stores user records and facial encodings used during recognition. Gesture data is collected as landmark-based samples and used in a separate training pipeline to produce a trained gesture model. During runtime, the system loads this model and combines it with live hand data to classify gestures. The final recognition results, including detected identity, gesture, confidence, and latency, are then returned to the frontend and displayed to the user.

Overall, this architecture was designed to support a responsive prototype. It separates the interface, processing logic, and data components clearly, while still allowing them to work together as one real-time system.

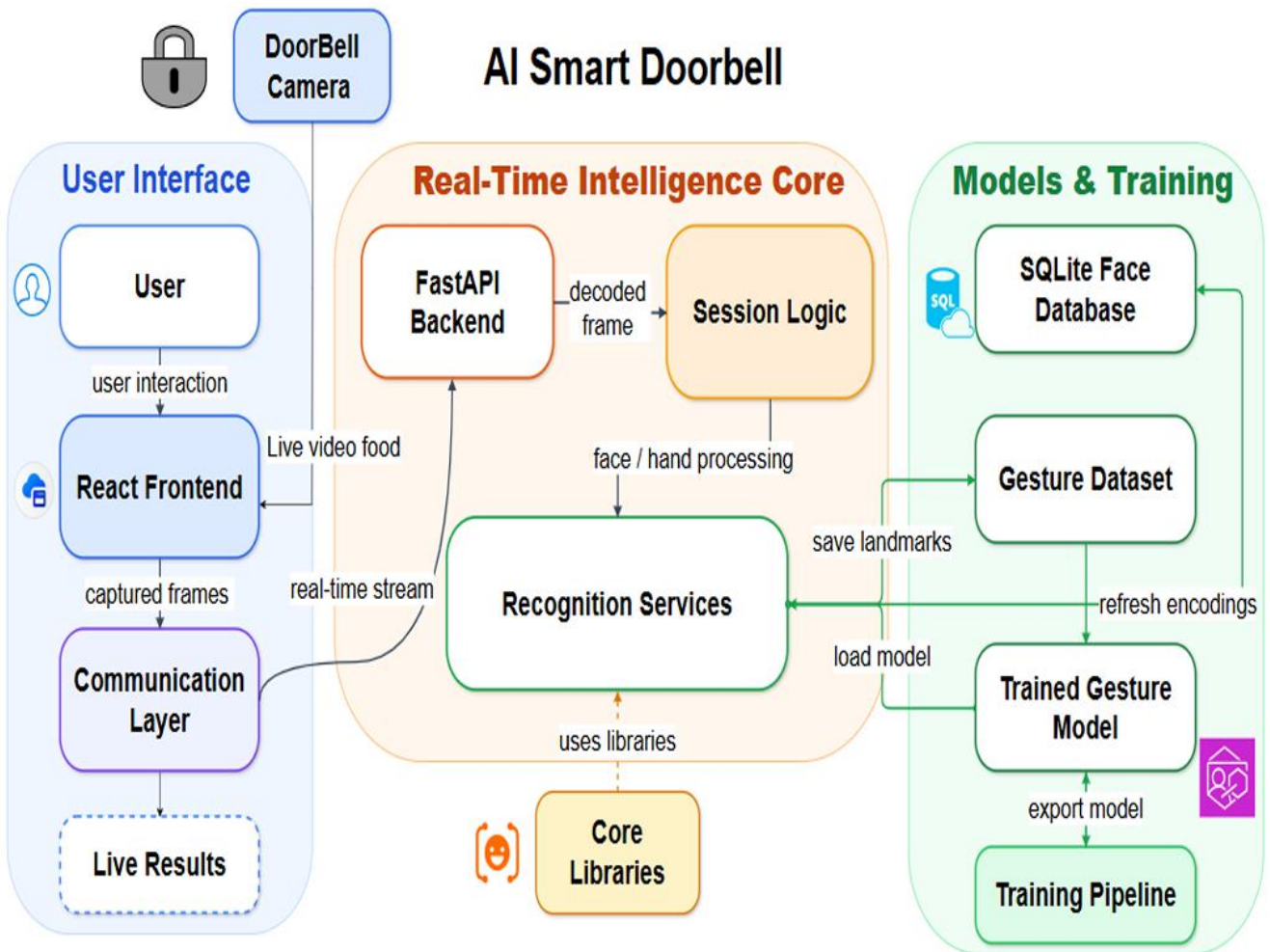
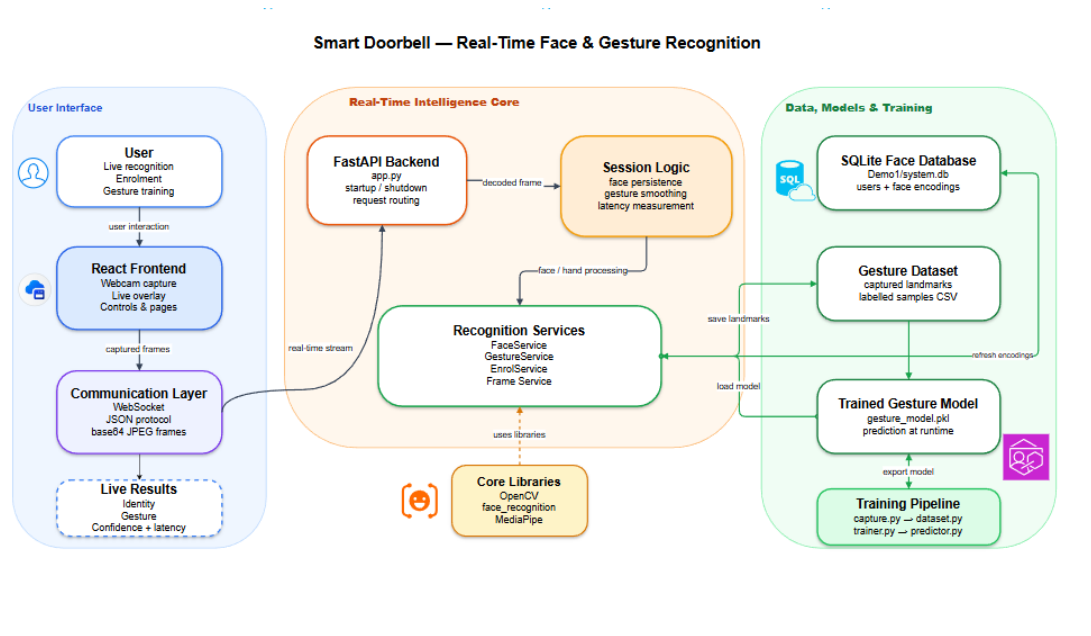


Figure 1: System architecture of the smart doorbell assistant



6 Project Plan

The project was developed in stages across the academic year using a practical approach. Rather than building the final web application structure first, the work began by focusing on the core technical features of the system. This made it possible to test whether the main project idea was achievable before moving on to full system integration. Jira was used to organise the project into tasks and sprints, while GitHub was used throughout development for version control and code management.

The first stage of the project involved defining the problem area and refining the project idea. The overall aim was to develop a smart doorbell assistant that could help older people, particularly those living with Alzheimer's disease or memory difficulties, recognise familiar visitors at the door. Once this goal had been established, the project moved into early technical experimentation to determine whether facial recognition and gesture recognition could be developed as the main features of the prototype.

The next stage focused on getting the face recognition and hand gesture recognition working as separate core components. This was an important part of the project because both of these features needed to be understood and tested before they could be integrated into a full application. Face recognition work involved detecting and identifying known individuals using stored facial encodings, while gesture recognition involved exploring hand landmark detection and training a model to classify selected gestures. At this point, the work was more experimental and focused on proving that the recognition pipeline could function reliably.

Once these recognition features had been explored, the project moved on to the database side of the system. A SQLite database was introduced to store user data and facial encodings so that known individuals could be enrolled and recognised later during runtime. This stage was

important because it turned the face recognition feature from a simple test into something more practical and reusable within the wider prototype.

After the recognition features and database were in place, attention turned to building the full application structure around them. This involved developing the backend using FastAPI and creating a frontend using React. A WebSocket-based communication pipeline was then introduced so that webcam frames captured on the frontend could be sent to the backend continuously for real-time processing. This allowed the previously developed face and gesture recognition components to be connected into a single live system. As part of the gesture recognition work, a Random Forest model was trained using captured hand landmark data so that selected gestures could be classified during runtime. This was an important step in moving from simple hand tracking to a more complete gesture recognition pipeline.

The latter stages of the project focused on integration, refinement, and testing. Once the frontend, backend, database, and recognition modules had been connected, the main goal was to improve stability and overall performance. This included working on features such as session logic, smoothing, and latency measurement so that recognition results could be displayed more consistently in real time. Additional work was also carried out on enrolment and live result display so that the prototype would feel more complete from a user point of view.

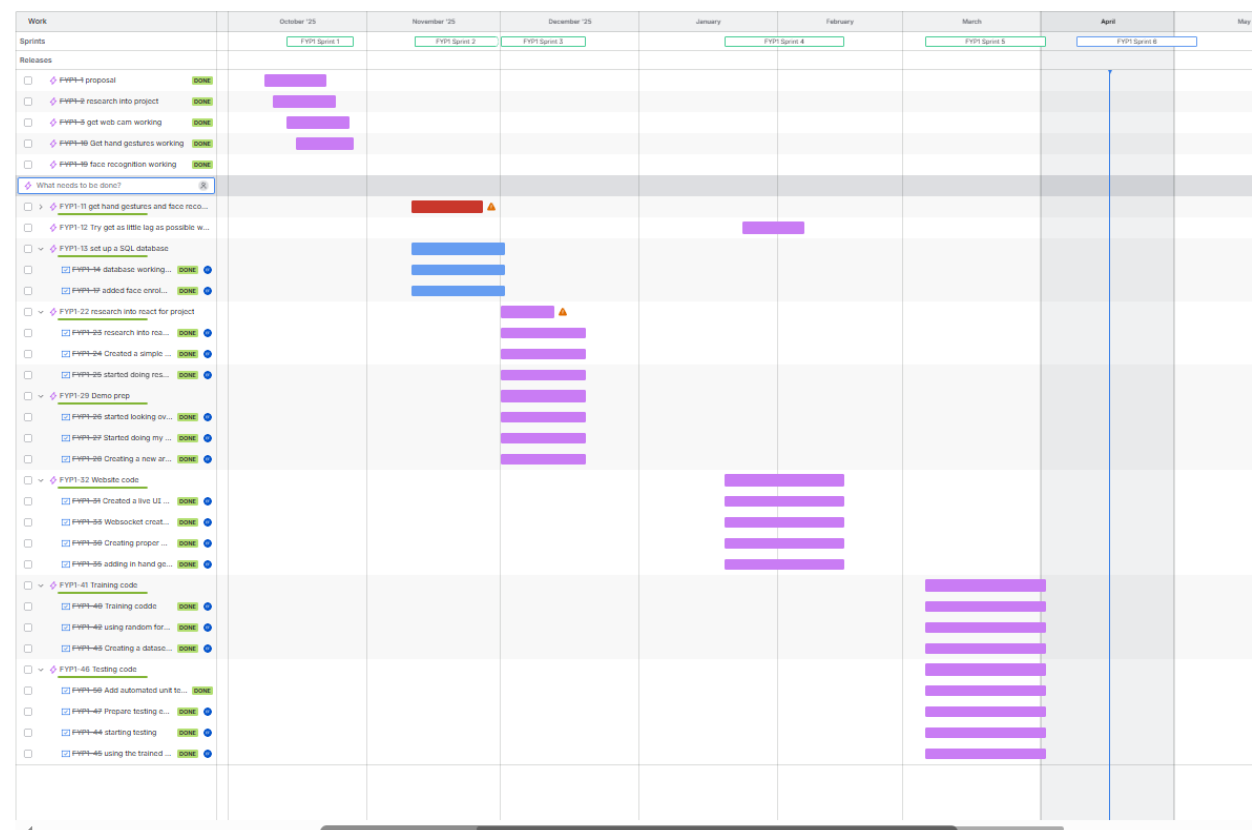


Figure 2: Jira sprint board used to plan and track project development

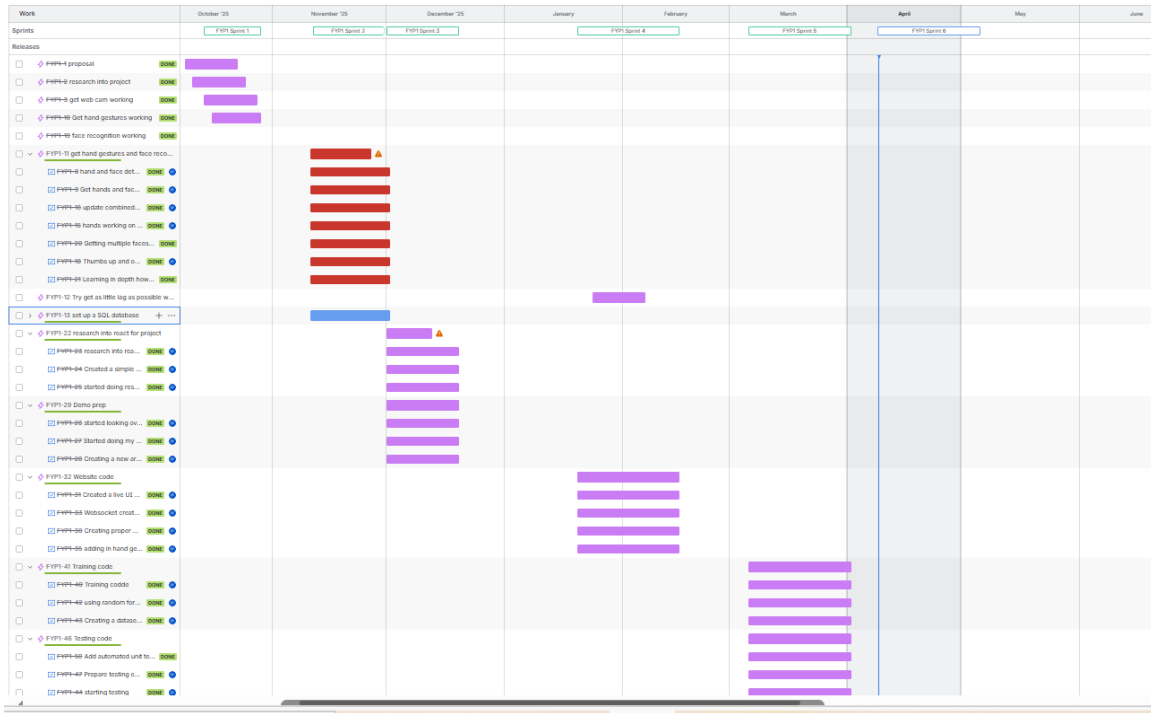
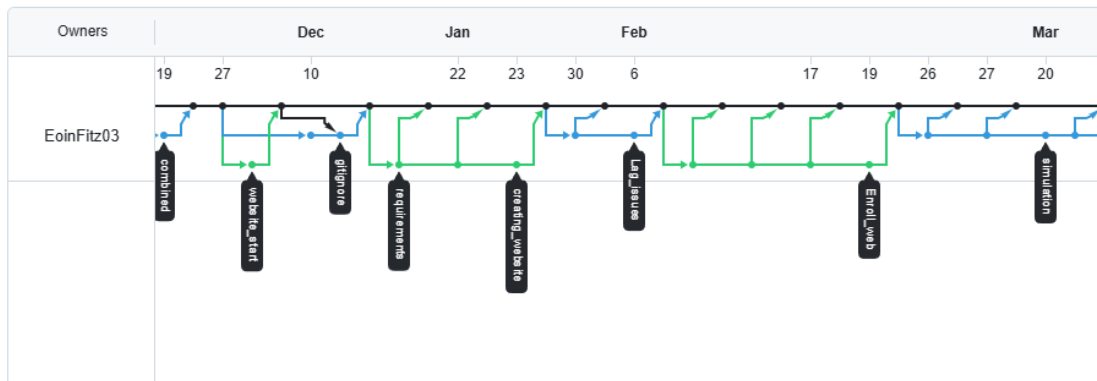


Figure 2 shows the Jira planning board used during the project. This was used to organise the development into stages and track progress across key areas such as early research, recognition features, database work, frontend and backend integration, training, and testing. It also helped break the project into smaller tasks and supported a more structured workflow across the year.



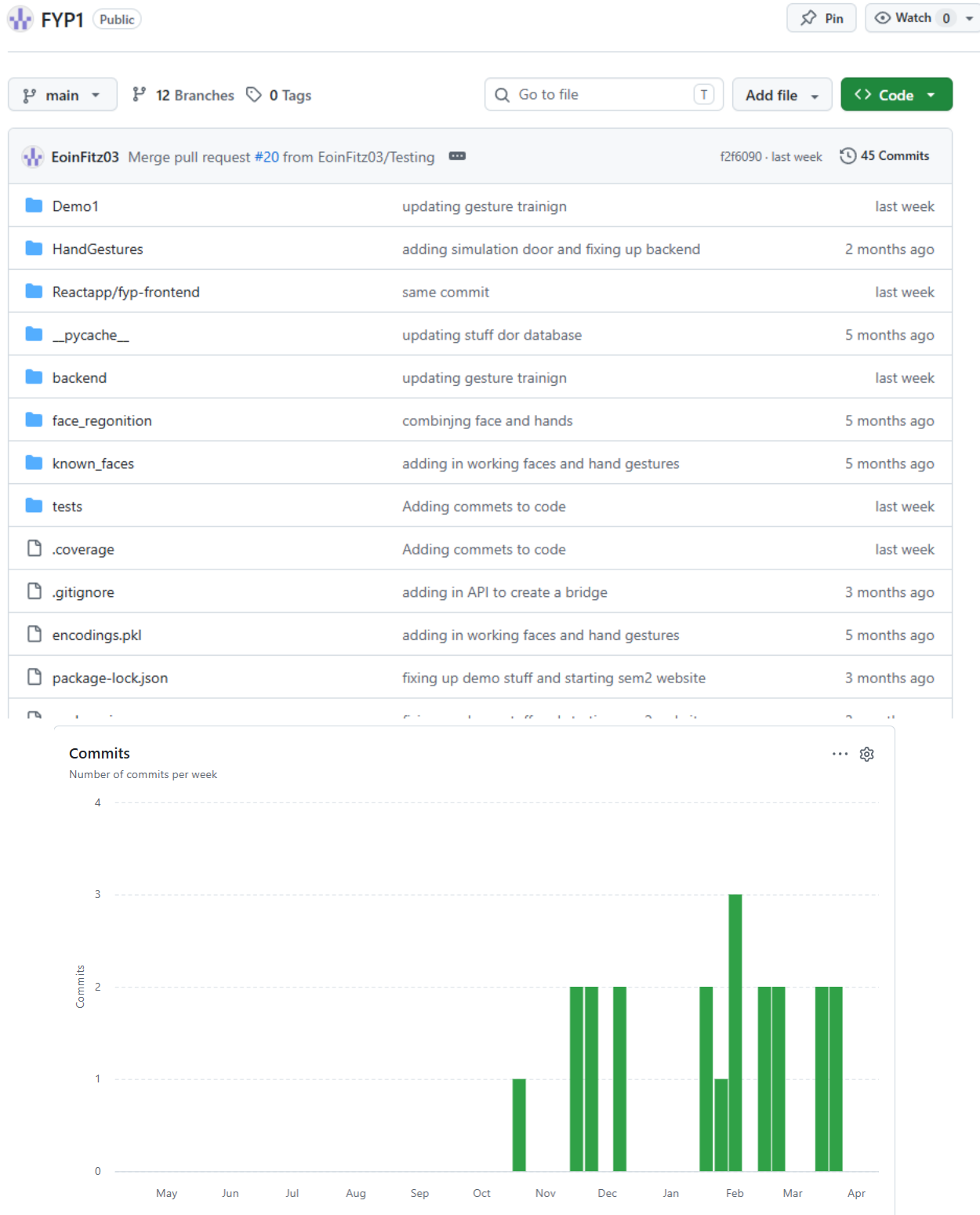


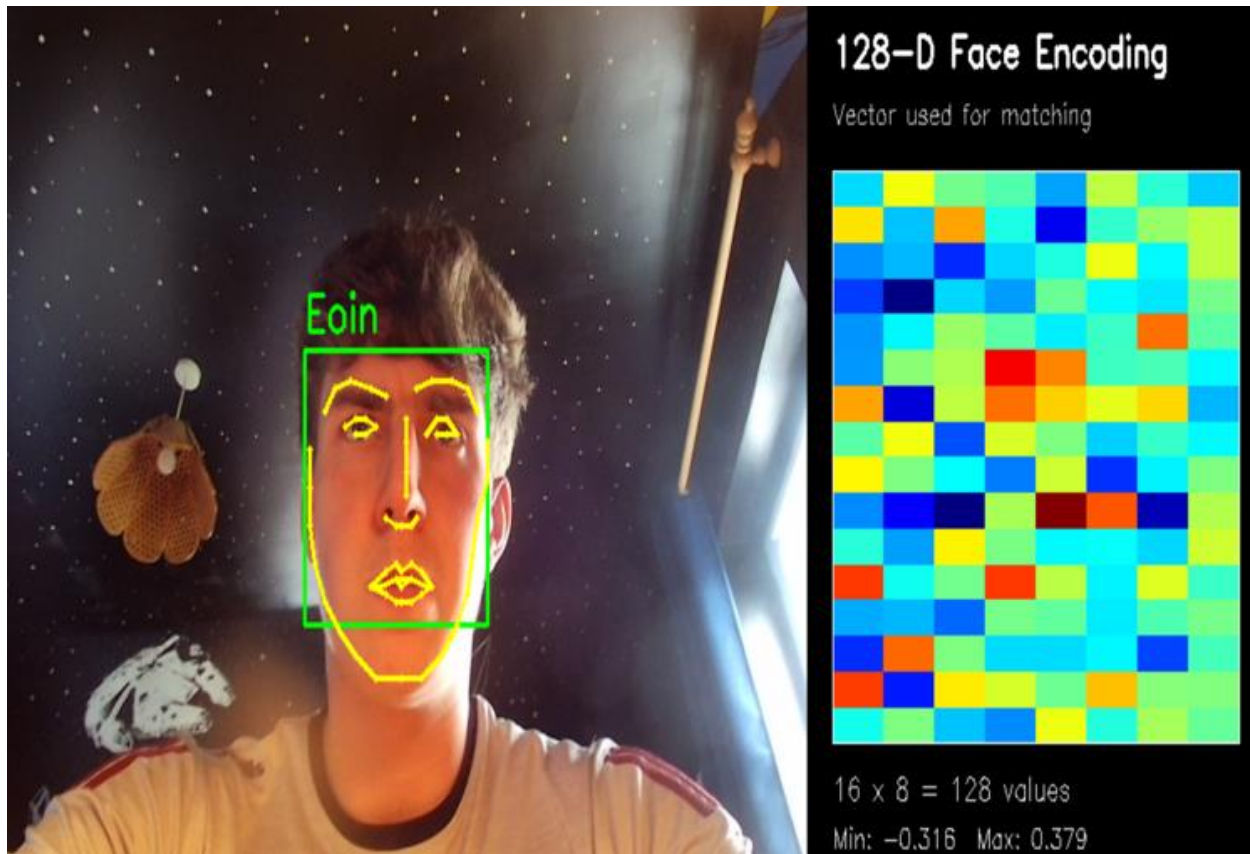
Figure 3: GitHub repository activity showing ongoing implementation and version control

Figure 3 shows GitHub repository activity during the project. GitHub was used as the main version control platform and provided a record of code changes, implementation progress, and feature development over time. This supported a more organised development process and made it easier to manage updates as the project moved from separate prototype components to one integrated system.

Overall, the project followed a staged but flexible plan. The work began with the core AI features, then expanded into database support, and finally developed into a complete web-based prototype through frontend and backend integration. This approach was useful because it reduced risk early on and ensured that the main technical parts of the project were functioning before time was spent on the full application interface.

7 Implementation

7.1 Face Recognition



Face recognition was one of the main technical features developed in this project and formed the foundation of the smart doorbell assistant. Since the main purpose of the system was to help older users recognise familiar people at the door, it was important that the project could identify known individuals accurately and return a result in real time. For that reason, face recognition was one of the earliest parts of the project to be researched and tested.

The face recognition pipeline used the `face_recognition` library built on `dlib`. Frames were downscaled by a factor of 0.5 before processing to reduce computational cost, and face detection used the HOG (Histogram of Oriented Gradients) model. A matching tolerance threshold of 0.50 was applied meaning any face with a distance score below 0.50 from a stored encoding was considered a match. Face encodings are 128-dimensional numerical vectors stored as JSON in the SQLite database. To reduce flickering in the live display, a recognised face was persisted for up to 800ms after detection was lost before resetting to Unknown. Recognition was also run only every 2 frames rather than on every frame, reducing processing cost without significantly affecting the response.

The face recognition process began by capturing facial data for known users and generating facial encodings from these images. A facial encoding is a numerical representation of the important facial features in an image, which can later be compared against other encodings to determine whether two faces are likely to belong to the same person. These encodings were stored so that they could be reused during live recognition rather than recalculated each time the system was run.

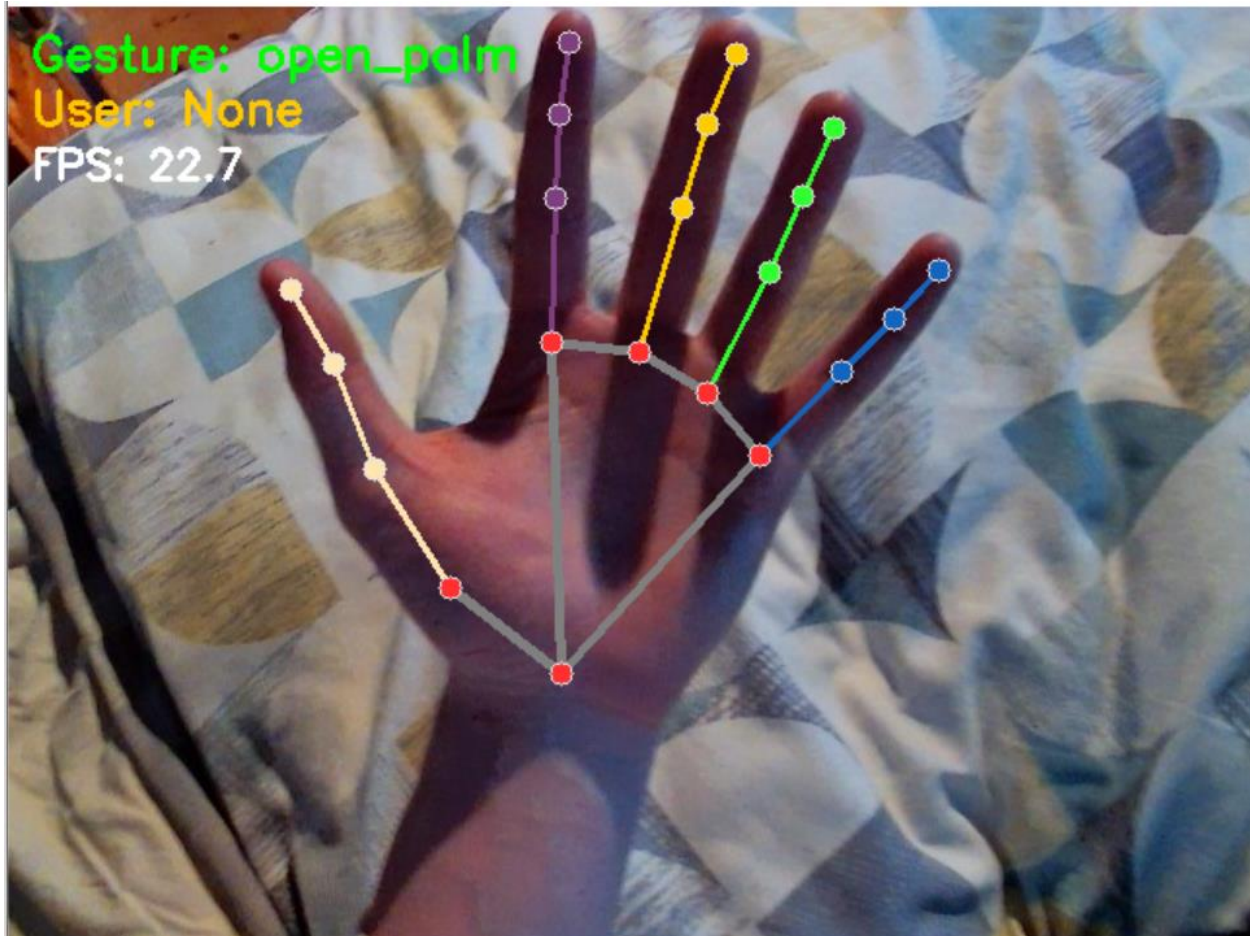
During runtime, frames captured from the live webcam feed were processed by the recognition pipeline. The system first detected whether a face was present in the incoming frame and then generated an encoding for that detected face. This live encoding was compared against the stored encodings of enrolled users. If the comparison score was within a suitable threshold, the system returned the matched person. If no suitable match was found, the system labelled the face as unknown. This allowed the system to distinguish between familiar visitors, such as family members or carers, and people who had not been enrolled.

An important part of the implementation was linking face recognition to the database so that recognised users could be managed more practically. Rather than relying only on hardcoded images, the project used stored user records and facial encodings to make the recognition system more flexible. This also supported the enrolment feature, where new users could be added to the system and recognised in future sessions.

Because the project was designed as a real-time prototype, performance and stability were also important. Face recognition results can vary slightly from frame to frame due to changes in lighting, camera angle, or face position. To improve consistency, the system used session-based logic and smoothing so that recognition results were more stable and did not flicker unnecessarily between recognised and unknown states. This helped make the output more reliable for the user interface.

Overall, the face recognition component was a key part of the project because it provided the core assistive function of the smart doorbell assistant. It allowed the system to identify known visitors from a live camera feed and display that information back to the user in real time. This made it possible for the prototype to support its main aim of helping older users recognise familiar people at the door.

7.2 Gesture Recognition and Model Training



Gesture recognition was developed as a supporting feature in the smart doorbell assistant to allow simple interaction after a visitor had been recognised. While facial recognition was the main feature used to identify who was at the door, gesture recognition added another layer to the prototype by allowing the user to simulate door-response actions through hand movements. In the context of the project, this was designed as a simple and contactless way for the user to respond to recognised visitors such as family members or carers.

The gesture recognition process began by detecting and tracking hand landmarks from webcam input. Instead of working directly with raw images alone, the system used landmark points taken from the detected hand. These landmarks represented important positions on the hand, such as finger joints and fingertip locations, and provided a more structured way to describe hand shape and movement. Using landmarks made the data more suitable for training a classification model.

To build the gesture recognition system, samples of hand landmark data were collected for a set of selected gestures. These samples formed the training dataset used in the machine learning stage of the project. Once enough labelled examples had been gathered, a Random Forest classifier was trained to distinguish between the chosen gestures. This model allowed the system to move beyond simple hand tracking and into actual gesture classification.

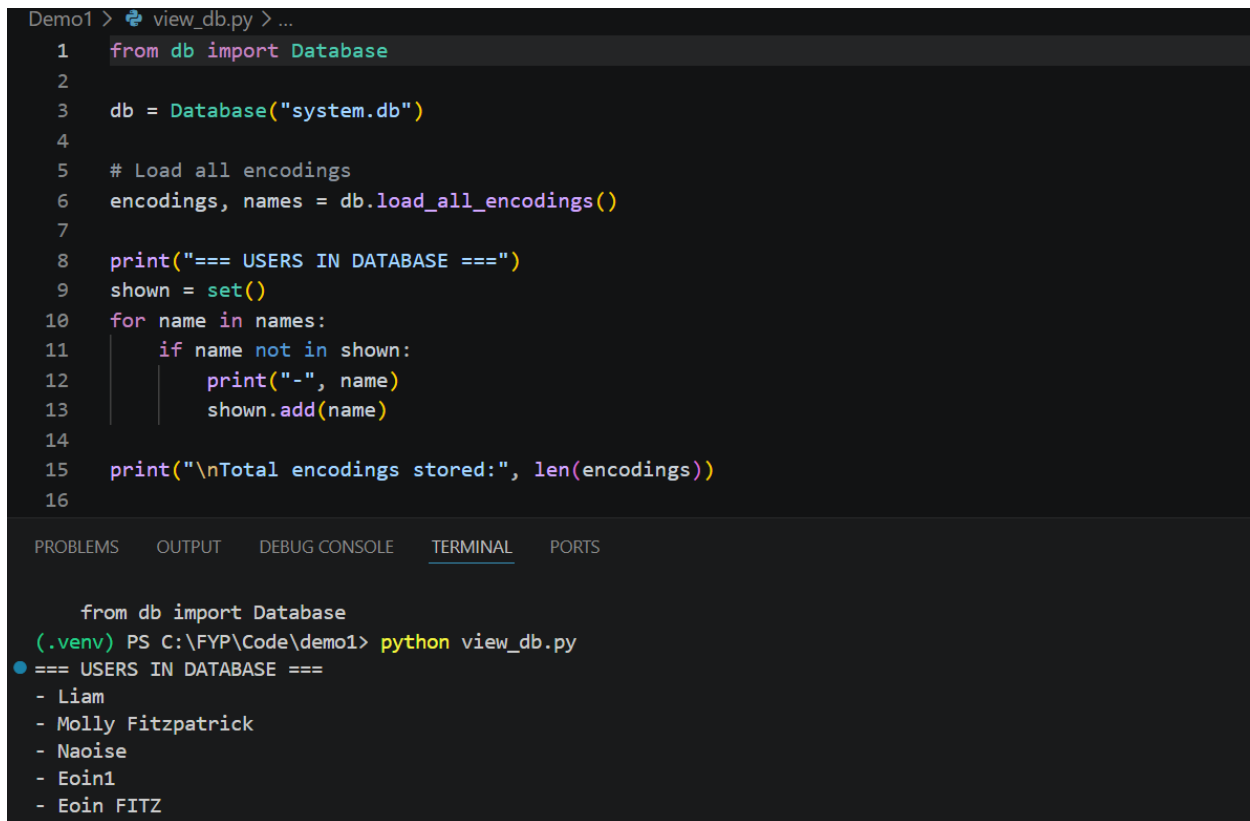
The trained model was then integrated into the runtime pipeline. During live use, hand landmarks were extracted from incoming webcam frames and passed to the trained classifier, which predicted the most likely gesture. The predicted result could then be linked to a simulated door-response action within the prototype, such as opening or locking the door for a recognised visitor. As this project was a prototype, these actions were simulated in software and were not connected to a physical smart lock.

The gesture model was trained using a Random Forest classifier with 200 decision trees, no maximum depth constraint, and a minimum of 1 sample per leaf node. Training used an 80/20 train-test split. The feature set consisted of 63 landmark coordinates x, y, z values for each of 21 hand keypoints detected by MediaPipe, plus a hand-side encoding (Left/Right/Unknown) giving 64 features in total. During live use, gesture frames were resized to a width of 320 pixels before processing, and predictions were smoothed using a majority vote across a 5-frame sliding window, requiring a minimum of 2 votes before a gesture result was confirmed.

As with face recognition, stability was an important issue in gesture recognition. Live predictions can vary between frames due to hand position, camera quality, lighting, or brief tracking errors. To improve reliability, smoothing logic was used so that the system did not react too quickly to inconsistent frame-by-frame predictions. This helped make the gesture output more stable and reduced the chance of incorrect simulated actions being triggered.

Overall, the gesture recognition and model training stage added an important interactive element to the project. It showed how machine learning could be used not only to detect hand movement, but also to classify meaningful gestures in real time. Within the smart doorbell assistant prototype, this feature supported the wider aim of creating a more usable and assistive system for older people by allowing simple gesture-based simulated responses once a visitor had been identified.

7.3 Database Design



```

Demo1 > view_db.py > ...
1  from db import Database
2
3  db = Database("system.db")
4
5  # Load all encodings
6  encodings, names = db.load_all_encodings()
7
8  print("=== USERS IN DATABASE ===")
9  shown = set()
10 for name in names:
11     if name not in shown:
12         print("-", name)
13         shown.add(name)
14
15 print("\nTotal encodings stored:", len(encodings))
16

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

      from db import Database
(.venv) PS C:\FYP\Code\demo1> python view_db.py
• === USERS IN DATABASE ===
- Liam
- Molly Fitzpatrick
- Naoise
- Eoin1
- Eoin FITZ

```

The database was an important part of the project because it allowed the recognition system to move beyond simple testing and become a more practical prototype. Instead of relying on temporary images, the project used a database to store information about enrolled users and their associated face encodings. This made it possible for known individuals to be added to the system and recognised again in the future.

A SQLite database was used in this project because it was lightweight, easy to integrate, and suitable for my project. Since the system was designed as a local real-time prototype rather than a large-scale deployed product, SQLite provided a simple way to manage data without needing a separate database server. This made development easier while still allowing the project to demonstrate proper data storage.

The database consisted of three tables. The users table stored one row per enrolled person, containing an ID, name, role, and creation timestamp. The face_encodings table stored one row per captured face sample, with the 128-dimensional encoding serialised as a JSON string. The events table logged recognition activity over time, storing the recognised user name, detected gesture, simulated action, and timestamp. This structure allowed multiple face encodings to be stored per user, improving recognition robustness by averaging across slightly different captures of the same person.

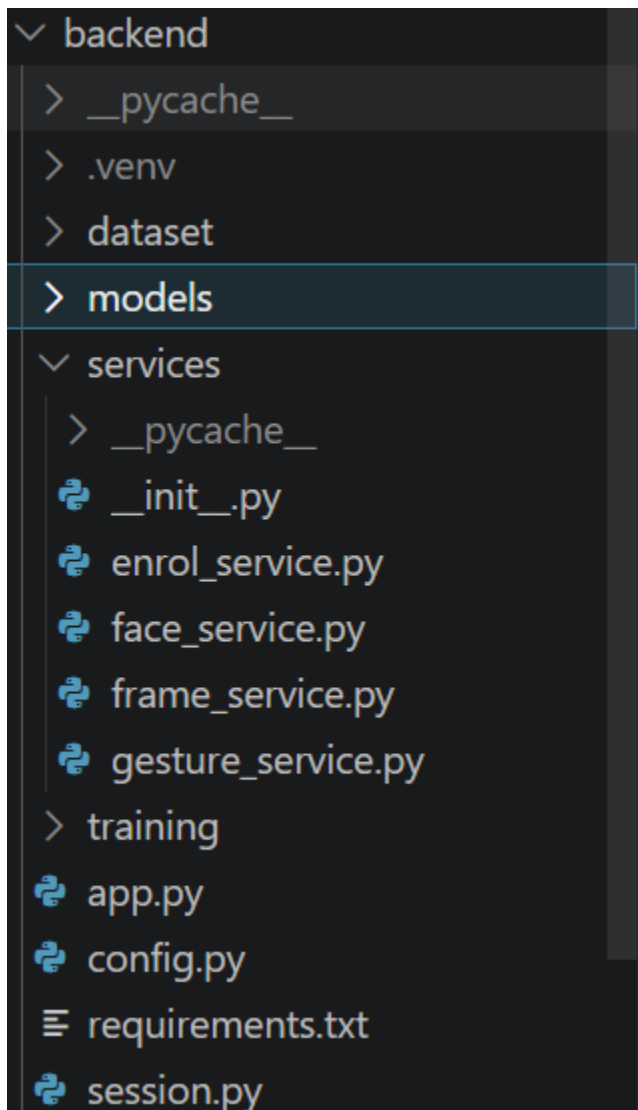
The main purpose of the database was to store user records linked to face recognition data. When a new person was enrolled, their details and facial encoding could be saved so that the system could later compare live webcam input against the stored faces. This supported the core use of the smart doorbell assistant, where familiar visitors such as family members or carers could be recognised in real time. By keeping this information in a database, the system became more flexible and reusable.

The database also supported the enrolment feature of the project. Enrolment was important because it allowed the dataset of known users to grow over time instead of being fixed from the start. This meant that the prototype could simulate a more realistic home-use scenario, where additional people could be registered as trusted visitors when needed. It also helped connect the recognition system to the wider application, as the stored data could be accessed by backend services during live processing.

From a design point of view, the database acted as a bridge between the development stage and the final integrated system. The face recognition feature depended on accurate stored encodings, while the backend needed a structured way to retrieve user records during runtime. Without the database, the prototype would have been much more limited and less realistic. Its inclusion made the project more complete by supporting the storage, enrolment, and repeated recognition.

Overall, the database design played a key role in turning the project into a usable prototype. It provided a structured way to store recognised user information, supported the enrolment process, and allowed the face recognition pipeline to operate in a more practical way within the smart doorbell assistant.

7.4 Backend Development



```

(.venv) PS C:\FYP\Code\backend> uvicorn app:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\\FYP\\Code\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [16364] using WatchFiles
C:\FYP\Code\backend\.venv\Lib\site-packages\face_recognition_models\__init__.py:7: UserWarning: pkg_resources
eprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources packag
ted for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  from pkg_resources import resource_filename
INFO: Started server process [7228]
INFO: Waiting for application startup.
[FaceService] Loaded 6 encodings from C:\FYP\Code\Demo1\system.db
[backend] MediaPipe Hands initialised

```

The backend formed the main processing layer of the smart doorbell assistant and was responsible for handling communication, recognition logic, and data access during runtime. Once the core face recognition, gesture recognition, and database components had been developed, the

backend was built to connect these parts into one working system. A FastAPI-based backend was used in the project because it provided a clear structure for routing, service organisation, and real-time communication.

One of the most important parts of the backend was the WebSocket connection between the frontend and backend. Since the system was designed to work in real time, a normal request-response model like REST, would not have been as suitable for continuously sending webcam frames and receiving recognition results. Instead, the frontend streamed image frames to the backend through a WebSocket connection, allowing the system to process incoming data continuously and return results without needing repeated separate requests. This made the application more responsive and better suited to a live doorbell.

The backend exposed a WebSocket endpoint at /ws for the live recognition stream, an enrolment endpoint for capturing and saving new face samples, and a training capture endpoint for recording gesture landmark data. The application was built using FastAPI with CORS middleware configured to allow requests from the React frontend running on port 5173. On startup, the backend loaded all known face encodings from the database into memory and initialised the MediaPipe Hands detector. Face recognition was run every 2 frames and gesture recognition every 3 frames, with the recognition results returned to the frontend in a JSON payload containing the recognised person, confidence score, Euclidean distance, gesture label, gesture confidence, and frame latency in milliseconds.

Within the backend, the application logic was divided into separate service areas to keep the code more organised and modular. These services handled tasks such as face recognition, gesture recognition, enrolment, and general frame processing. This approach made the project easier to manage as it grew, as each part of the backend could focus on a specific responsibility rather than placing all of the logic in a single file. It also made testing and debugging easier, since individual services could be worked on separately.

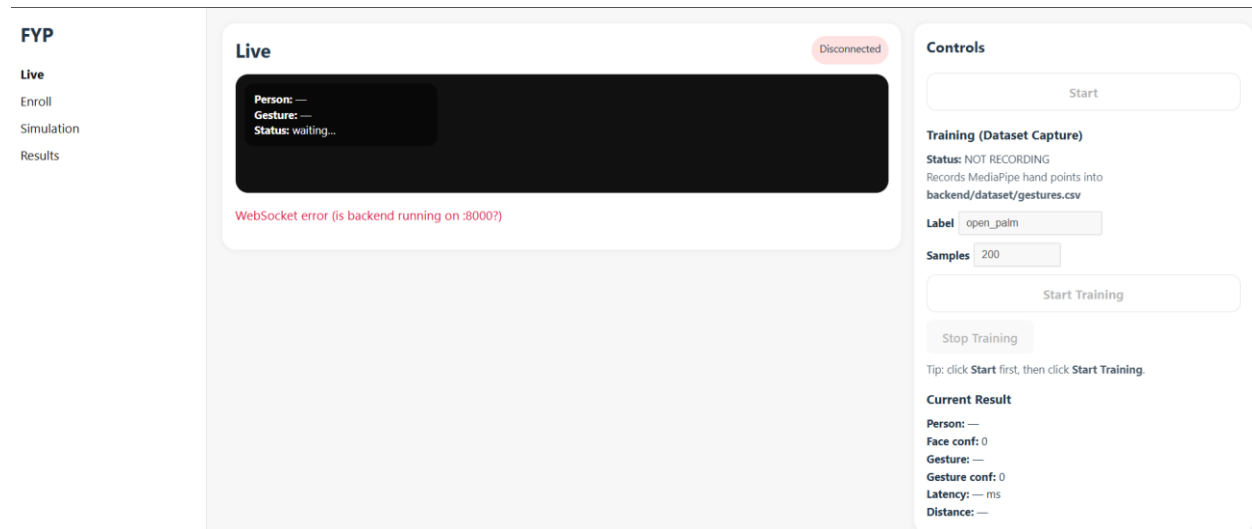
The backend was also responsible for session-related logic used to improve stability during runtime. Since both face recognition and gesture recognition can produce slightly inconsistent results across multiple frames, the backend included support for smoothing and persistence to make the output more stable. This helped reduce flickering between recognised and unknown states and made gesture predictions less sensitive to short-term change. In addition, latency measurement was included so that the performance of the real-time pipeline could be monitored during development.

Another important role of the backend was managing communication with the database and trained model files. During runtime, the backend loaded stored facial encodings from the database and used them when comparing live webcam frames against enrolled users. It also loaded the trained gesture model so that live hand landmark data could be passed through it for gesture prediction. This made the backend the central point where live input, stored data were brought together.

Overall, the backend development stage was essential in turning separate project components into a connected real-time application. It provided the processing pipeline that linked the live

frontend camera feed to the recognition services, the stored database records, and the trained gesture model. As a result, the backend acted as the intelligence of the smart doorbell assistant prototype and played a major role in making the system function as a complete interactive system.

7.5 Frontend Development



The frontend was developed to provide the main user interface for the smart doorbell assistant and to allow the different system features to be used in a clear and practical way. While the backend handled the main recognition and processing tasks, the frontend was responsible for capturing live webcam input, displaying recognition results, and supporting user interaction with the prototype. A React-based frontend was used in this project because it allowed the interface to be structured into reusable components and supported the development of a responsive web-based application.

One of the most important roles of the frontend was capturing the live video feed from the webcam. This live feed represented the doorbell camera input within the prototype and formed the starting point of the real-time recognition process. Frames from the webcam were encoded and sent to the backend through the WebSocket connection, where they could be processed for face recognition and gesture recognition. This made the frontend a key part of the real-time system rather than only a display layer.

The frontend was also responsible for presenting live recognition results back to the user. When a face was recognised, the returned result could be displayed on the interface so that the user could see who was at the door. Gesture outputs and related simulated door actions could also be shown as part of the live feedback. This was important for the overall purpose of the project, as the system needed to provide information in a simple and understandable way for the user rather than only processing data in the background.

In addition to the live recognition view, the frontend also supported other parts of the project such as enrolment and gesture training workflows. These views helped make the prototype more

complete by allowing users to move between the main features of the system through a single interface. From a design point of view, this was important because it connected the technical backend features to a usable application structure.

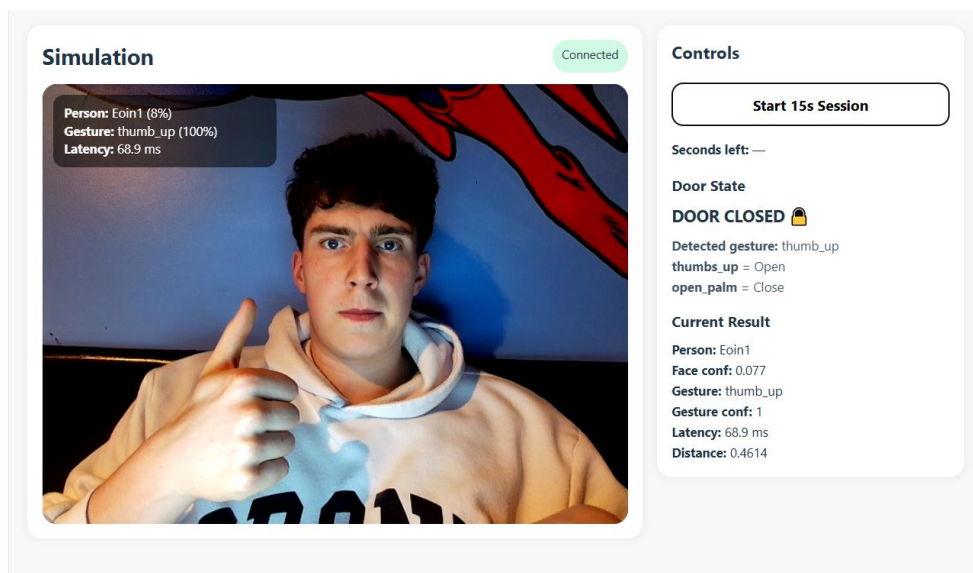
The frontend also played a role in the overall user experience of the prototype. Since the project was aimed at an assistive home-use scenario, it was important that the interface remained straightforward and not overly complex. The live display, overlays, and controls were designed to support the main functions of the system without distracting from the core goal of recognising visitors and providing simple simulated responses.

Overall, the frontend development stage was important because it turned the project into an interactive application rather than just a collection of backend scripts and machine learning components. By capturing live video, sending data for processing, and displaying results in real time, the frontend allowed the smart doorbell assistant prototype to be used as a complete web-based system.

7.6 System Integration

FYP

Live
Enroll
Simulation
Results



System integration was the stage where the separate parts of the project were brought together into one complete smart doorbell assistant prototype. Earlier stages of the project focused on developing the main features individually, including face recognition, gesture recognition, the database, the backend, and the frontend. However, for the prototype to work properly in a real-time setting, these parts needed to operate as one connected system rather than as separate components.

The integration process began by connecting the React frontend to the FastAPI backend through a WebSocket communication pipeline. This allowed webcam frames captured on the frontend to be sent continuously to the backend for processing. Once this connection was working reliably, the backend could apply the previously developed face recognition and gesture recognition logic to the incoming frames and return the results back to the interface in real time.

A key part of this stage was linking live recognition to stored project data. The backend loaded facial encodings from the SQLite database so that faces detected in live webcam frames could be compared against enrolled users. At the same time, the trained Random Forest gesture model was loaded so that hand landmark data could be classified during runtime. This meant that both stored face data and the trained gesture model became part of the same live processing pipeline.

The system was designed so that facial recognition acted as the main assistive feature, while gesture recognition provided a supporting interaction method. When a recognised visitor, such as a family member or carer, appeared in front of the webcam, their identity could be displayed on the frontend. Gesture input could then be used to trigger simulated door-response actions within the prototype. In this way, the integrated system supported both recognition and interaction in a single workflow.

One of the main challenges during integration was maintaining stable real-time performance. Since the project involved continuous frame capture, backend processing, model inference, and live result display, it was important to manage timing and reduce inconsistent results. This was addressed through session-based logic, smoothing methods, and latency tracking, which helped make the output more stable and improved the overall responsiveness of the prototype.

From an implementation point of view, system integration was the stage that turned the project from a set of technical features into a complete application. It demonstrated that the face recognition pipeline, gesture classification model, database storage, backend services, and frontend interface could all work together in one real-time assistive system. This was a significant part of the project, as it showed that the original idea could be developed into a functioning prototype rather than remaining as separate experiments.

Overall, the integration stage brought together all of the major technical elements of the project into one working system. It was this stage that allowed the smart doorbell assistant to function as intended. Capturing live input, recognising familiar visitors, classifying gestures, and displaying useful results back to the user in real time.

8 Testing and Evaluation

8.1 Testing Approach

Testing was an important part of this project because the smart doorbell assistant was designed as a real-time interactive prototype rather than a static software application. Since the system combined facial recognition, gesture recognition, database storage, backend services, and a web-based frontend, it was necessary to test both the individual components and the behaviour of the system. The main aim of testing was to check that the core features worked correctly, operated reliably and supported the overall purpose of the project.

A combination of different testing approaches was used during development. At component level, individual features such as face recognition, gesture classification, enrolment, and backend services were tested to confirm that they behaved as expected. This helped identify problems early and made it easier to fix issues within a specific part of the system before moving on to full integration. In addition, integration-style testing was important because the project depended on multiple technologies working together in a single pipeline.

The testing process focused on the most important functional areas of the prototype. These included recognising known users from a live webcam feed, correctly identifying unknown users, classifying selected hand gestures, storing and retrieving enrolled user data, and maintaining stable communication between the frontend and backend. Because the project operated in real time, testing also considered factors such as result stability, response consistency, and general usability during live use.

As the system developed, testing was also used as part of refinement. Early results showed that live recognition could vary between frames, particularly in cases involving changes in lighting, hand position, or camera angle. This meant that testing was not only used to confirm whether a feature worked, but also to improve how reliably it worked in practice. Features such as smoothing, session logic, and latency tracking were therefore considered as part of the testing and evaluation process.

Overall, the testing approach used in this project aimed to evaluate both technical correctness and practical performance, which was important for a real-time assistive prototype such as this one.

8.2 Face Recognition Testing

Face recognition testing was carried out to evaluate whether the system could correctly identify known users from the live webcam feed and distinguish them from unknown individuals. Since this was the core feature of the smart doorbell assistant, it was important to test not only whether recognition worked, but also whether it worked consistently enough to be useful in a real-time.

The main focus of this testing was on recognised and unrecognised cases. Known users who had been enrolled in the system were presented to the webcam so that the system could compare the live facial encoding against the stored encodings in the database. If the similarity score was within the selected threshold, the system returned the correct identity. Testing also included unknown users so that the system could be checked for its ability to avoid incorrectly assigning a known identity where no valid match existed. This was important because false recognition would reduce trust in the system.

Testing showed that the face recognition pipeline was able to identify enrolled users successfully when the face was clearly visible and the webcam conditions were reasonable. Recognition was generally strongest when the person was positioned directly in front of the camera and lighting conditions were stable. In cases where the face was partially turned, too far from the camera, or affected by poor lighting, recognition became less stable. This was expected, as live face recognition performance can vary depending on image quality and viewing conditions.

One issue observed during testing was that recognition results could fluctuate between consecutive frames. For example, a user might be identified correctly in one frame and then briefly appear as unknown in the next. This type of behaviour is common in live recognition systems and can happen because of small changes in angle, facial expression, or frame quality. To reduce this effect, smoothing and session-based persistence were introduced so that the displayed result would remain more stable over time rather than reacting instantly to every frame. This improved the overall usability of the system and made the output more reliable from the user's point of view.

The face recognition testing also helped confirm that linking the recognition pipeline to the database was working correctly. Enrolled users could be added to the system and then recognised again during live use, which showed that the stored facial encodings were being loaded and applied properly during runtime. This was an important part of the overall prototype, as it demonstrated that the recognition feature could support repeated real-world style use rather than only one-off tests.

Overall, face recognition testing showed that the prototype was capable of carrying out its main task of identifying familiar visitors in real time. While performance was still affected by some factors such as lighting and camera position, the system was able to recognise enrolled users successfully and provide useful live feedback. This confirmed that the face recognition component was suitable as the main assistive feature of the smart doorbell assistant prototype.

8.3 Gesture Recognition Testing

Gesture recognition testing was carried out to evaluate whether the system could correctly detect and classify the selected hand gestures during live use. Although gesture recognition was a supporting feature rather than the main assistive function of the prototype, it still played an important role by allowing the user to simulate door-response actions for recognised visitors. Because of this, testing focused on both classification accuracy and the stability of gesture predictions in real time.

The gesture recognition pipeline was tested by presenting the trained gestures in front of the webcam and checking whether the system returned the expected predicted class. This involved using live hand landmark data captured during runtime and passing it through the trained Random Forest model. If the predicted label matched the intended gesture, the result was considered correct. This allowed the performance of the trained model to be evaluated within the real prototype environment rather than only during offline training.

Testing showed that the gesture recognition feature was capable of identifying selected gestures successfully when the hand was clearly visible and positioned well in front of the camera. Predictions were generally more reliable when the hand shape matched the training examples closely and when the webcam captured the landmarks clearly. This showed that the trained model was able to use the landmark data meaningfully and classify the chosen gestures during live use.

However, testing also showed that gesture predictions could vary between frames. Small changes in hand angle, movement speed, finger position, or camera visibility sometimes caused short incorrect predictions or unstable outputs. This was especially noticeable when the hand was moving into position or leaving the frame. Since the system was designed as a real-time prototype, reacting directly to every single frame would have made the output less reliable and could have caused incorrect simulated actions.

To address this, smoothing logic was used to stabilise the gesture results. Rather than immediately reacting to one predicted class, the system considered short-term prediction behaviour across multiple frames before displaying a final gesture result. This reduced flickering and helped prevent false gesture activations. As a result, the gesture feature became more suitable for use as a controlled simulated interaction within the prototype.

The testing process also helped confirm that the trained Random Forest model had been integrated successfully into the live system. The project therefore moved beyond simple hand tracking and demonstrated actual machine learning-based gesture classification within the smart doorbell assistant. Although the feature remained more sensitive to live conditions than face recognition, it still showed that gesture-based interaction could be added to the system in a meaningful way.

Overall, gesture recognition testing showed that the prototype could classify selected gestures in real time and use them to support simulated door-response actions. While the results were influenced by factors such as hand position and camera visibility, the feature worked well enough to demonstrate the concept.

8.4 Backend and Service Testing

In addition to live feature testing, backend and service testing was carried out to evaluate the reliability of the main processing logic used in the smart doorbell assistant. This was important because the project did not rely only on visible frontend behaviour, but also on a number of backend services working correctly behind the scenes. These services included face recognition, gesture recognition, frame handling, and enrolment-related logic. Testing these parts individually

helped improve confidence in the system and supported a more structured software engineering approach.

The purpose of backend testing was to check that the main service functions behaved as expected under normal conditions and that they returned correct outputs for the data they received. This was especially useful for parts of the project that were difficult to assess only through live manual testing. For example, it allowed specific service logic to be tested in isolation without needing the full webcam and frontend pipeline to be running at the same time.

Service-level testing was particularly useful in the enrolment, face recognition, frame processing, and gesture logic areas. These tests helped confirm that the backend could process data correctly, return valid results, and handle expected inputs in a controlled way. In practice, this reduced the chance of small service-level issues going unnoticed until later stages of integration.

Backend testing also highlighted which parts of the codebase had stronger test coverage than others. Some services, such as the enrolment and face-related logic, achieved strong coverage, while gesture-related services showed lower coverage due to the wider range of runtime conditions and decision paths involved. This reflected the fact that gesture recognition was one of the more variable and experimental parts of the system, with more complex live behaviour compared to simpler service functions.

Although full automated testing of every real-time scenario was not practical, the backend tests still provided useful evidence that the main service structure was functioning correctly. This supported the overall reliability of the prototype and showed that the project was not based only on manual trial and error, but also included formal software testing practices.

Overall, backend and service testing strengthened the quality of the project by checking important logic at code level and supporting the wider evaluation of the prototype. Combined with live feature testing, it helped show that the system was working not only from the user's point of view, but also at the service level underneath the main interface.

8.4.1

Coverage report: 85%				
Files Functions Classes				
coverage.py v7.13.5, created at 2026-03-27 14:25 +0000				
File ▲	statements	missing	excluded	coverage
backend \ services \ __init__.py	0	0	0	100%
backend \ services \ enrol_service.py	65	1	0	98%
backend \ services \ face_service.py	51	3	0	94%
backend \ services \ frame_service.py	11	0	0	100%
backend \ services \ gesture_service.py	81	27	0	67%
Total	208	31	0	85%

Figure 4: Backend service test coverage results

Figure 4 shows the backend service coverage results achieved during testing. Overall coverage reached 85%, with particularly strong coverage in the enrolment, face recognition, and frame processing services. Gesture service coverage was lower at 67%, reflecting the greater complexity and variability of the runtime gesture logic.

8.5 Overall Evaluation

Overall, the testing and development work carried out in this project showed that the smart doorbell assistant was successful as a working prototype. The system was able to combine facial recognition, gesture recognition, database storage, backend processing, and a web-based frontend into one integrated application. Most importantly, it demonstrated the main idea behind the project which is helping older people recognise familiar visitors at the door through a real-time assistive system.

The strongest part of the prototype was the face recognition feature. This provided the main assistive value of the project and was able to identify enrolled users successfully under suitable conditions. By linking this feature to stored user records in the database and returning live results

through the frontend, the system achieved its core objective in a practical way. This made the project more than a simple recognition script, as it functioned as a complete interactive prototype.

The gesture recognition feature also added value to the system by introducing an extra layer of interaction. Although it was more sensitive to live conditions such as hand position, movement, and camera visibility, it still demonstrated that trained gestures could be classified in real time and linked to simulated door-response actions. This gave the prototype a more complete feel and showed how machine learning could be used not only for recognition, but also for interaction.

The evaluation also showed that integration was one of the most important achievements of the project. A major challenge was not just developing each feature individually, but making them work together reliably in real time. The successful use of WebSockets, backend services, database storage, and smoothing logic showed that the project developed beyond separate experiments and into a functioning end-to-end system.

At the same time, some limitations were identified during testing. Recognition performance could still be affected by lighting conditions, camera angle, and the quality of the live input. Gesture recognition was also less stable than face recognition in some cases, particularly when the hand was not clearly visible or when movement between frames was inconsistent. These limitations are common in real-time computer vision systems, especially at prototype stage, and do not take away from the overall success of the project. Instead, they highlight areas where further refinement could improve robustness.

From a wider point of view, the project was successful because it addressed a meaningful real-world problem rather than only demonstrating technical features. The system was designed to support older people, particularly those living with Alzheimer's disease or memory difficulties, by helping them recognise who is at the door. This gave the project a clear assistive purpose and made the final prototype more relevant and practical.

Overall, the evaluation of the project shows that the main aims were achieved. The smart doorbell assistant functioned as a real-time prototype, demonstrated successful recognition and interaction features, and showed clear potential as an assistive technology concept. While there is still room for improvement in robustness and future development, the final outcome represents a strong and meaningful implementation of AI within a home support setting.

8.6 Limitations and Future Work

Although the prototype was successful in demonstrating the main idea of the project, it is important to be clear about what it actually proved and what it did not prove. The project showed that a real-time smart doorbell assistant could be built using facial recognition, gesture recognition, a local database, backend services, and a web-based frontend. It also showed that these parts could be integrated into one working prototype capable of recognising familiar users and supporting simulated door-response actions. In that sense, the project successfully proved the technical feasibility of the concept.

However, the project did not prove that the system is ready for real-world deployment in a home environment. The prototype was tested in controlled development conditions and with a limited number of users and scenarios. This means that, while the concept was shown to work, it has not yet been validated at the level that would be required for a real assistive product. A real deployment would need more extensive user testing, stronger reliability across different lighting and camera conditions, better handling of edge cases, and a much higher level of robustness overall.

This is particularly important because the intended user group includes older adults and potentially vulnerable users, including people living with Alzheimer's disease or memory difficulties. In this kind of setting, system errors could have a greater impact than they would in a standard consumer application. For example, a missed recognition, an incorrect identity, or an unstable gesture prediction could cause confusion and reduce trust in the system. Because of this, any future real-world version would need to be designed very carefully so that it supports the user without encouraging over-reliance on automation.

There are a number of realistic future improvements that could build on this project. One practical development would be to improve the recognition pipeline with more training data, more testing across varied environments, and more robust handling of unstable live input. Another realistic improvement would be to expand the user interface and move the system beyond a browser-based prototype into a mobile application. A mobile app version could make the system more accessible and practical by allowing recognition results, alerts, and visitor information to be viewed more easily on a phone or tablet. This could improve usability for carers or family members as well as for the main user.

Other future developments could include increasing the number of supported gestures, improving the quality of the frontend design, and integrating the system with smart home hardware. However, while hardware integration is possible as a future direction, it would also introduce much greater safety, reliability, and security requirements. For that reason, moving from simulated actions to a real lock-controlled system would need to be treated as a much more advanced stage of development rather than a simple next step.

Overall, the project provides a strong proof of concept, but it should still be understood as a prototype rather than a finished product. Its main value is that it demonstrates the potential of the idea and provides a solid technical foundation for future work. With further refinement, broader testing, and more careful real-world design, the system could be developed into a more practical assistive solution in the future.

9 Ethics

Ethical considerations were an important part of this project because the smart doorbell assistant was designed around facial recognition, personal data, and a potentially vulnerable user group. Although the system was developed as a prototype, the technologies involved still raise important questions about privacy, consent, data handling, reliability, and user wellbeing. Because of this, ethics had to be considered not only in terms of technical design, but also in terms of how the system might affect the people using it.

One of the main ethical issues in this project is privacy. Facial recognition systems rely on capturing and processing images of people's faces, which is sensitive personal data. In the context of this project, the system stores facial encodings for enrolled users so that they can be recognised later. Even though the project was only developed as a prototype, it was still important to treat this data carefully and to recognise that any real-world deployment would need clear rules around storage, access, and protection of user information.

Consent is also a major ethical consideration. Since the system is intended to recognise familiar visitors such as family members or carers, those individuals would need to be aware that their facial data is being collected and used for recognition purposes. It would not be ethically acceptable to store or process data without knowledge or permission. For that reason, the enrolment process in a real-world version of the system would need to ensure that informed consent is given before a person's facial data is added.

Another important issue is the reliability of recognition results. Facial recognition is not perfect, and errors can happen due to lighting, camera angle, poor image quality, or similarities between faces. In a project like this, an incorrect result could cause confusion rather than reduce it. This is especially important because the system is aimed at older users, including people who may have Alzheimer's disease or memory difficulties. A system intended to support such users should not encourage blind trust in automated results. Instead, it should be viewed as an assistive aid rather than a fully autonomous decision-maker.

The gesture recognition part of the system also raises ethical questions, although to a lesser extent. Since gestures are used to simulate door-response actions, incorrect classification could result in the wrong simulated action being displayed. In this prototype this does not create a physical safety risk because no real lock is controlled, but it still highlights the need for caution when linking recognition systems to actions. In a real deployment, any move from simulated actions to physical control would require much stronger safety checks and fail-safe design.

There is also an ethical responsibility in how the project is presented. Because the intended user group includes older adults and potentially vulnerable people, it is important not to overstate what the prototype can do. The system should be described honestly as a prototype that

demonstrates the concept of assistive recognition and simulated interaction. Presenting it as a complete real-world solution would be misleading and would ignore the limitations identified during testing.

At the same time, the project has a positive ethical motivation. The main purpose of the system is to support older people in a practical way by helping them recognise familiar visitors and feel more confident when responding to someone at the door. In that sense, the project aims to use AI in a supportive and human-centred way rather than for surveillance or control. This assistive focus is an important part of the ethical value of the work.

Overall, the ethical issues in this project centre on privacy, consent, reliability, and the responsible use of AI with vulnerable users. Although the system was developed only as a prototype, these considerations remain important and would become even more significant in any future real-world version. Recognising these ethical challenges is essential to evaluating the project properly and showing that the design of assistive technology must consider people as well as technical performance.

10 Conclusion

The aim of this project was to design and develop a smart doorbell assistant that could help older people, particularly those living with Alzheimer's disease or memory difficulties, recognise familiar visitors at the door. To achieve this, the project brought together facial recognition, gesture recognition, database storage, backend processing, and a web-based frontend into a single real-time prototype. The final outcome was a working system that demonstrated how these technologies could be combined in a practical and meaningful assistive application.

The most important achievement of the project was the development of a functioning face recognition pipeline that could identify enrolled users from a live webcam feed and return the result in real time. This provided the core assistive feature of the system and directly supported the main purpose of helping the user recognise who was at the door. The project also successfully included a gesture recognition feature, supported by a trained Random Forest model, which allowed simulated door-response actions to be linked to recognised visitors. Although this part of the system was more sensitive to live conditions, it still added an important interactive element to the prototype.

Another key outcome of the project was the successful integration of all major components into one complete system. Rather than remaining as separate technical experiments, the face recognition, gesture recognition, database, backend, and frontend were connected through a real-time processing pipeline. This showed that the original project idea could be developed into a complete working prototype and not just a collection of isolated features.

Testing and evaluation showed that the system was capable of achieving its main objectives, while also highlighting practical limitations such as sensitivity to lighting, camera angle, and live gesture variation. These limitations are common in real-time computer vision systems and are reasonable at prototype stage. Despite them, the project clearly demonstrated the value of the concept and the feasibility of the chosen implementation.

Overall, this project was successful in delivering a smart doorbell assistant prototype with a clear assistive purpose. It showed how AI-based recognition technologies can be applied in a way that is not only technical, but also human-centred and socially relevant. The final system provides a strong proof of concept for how real-time recognition and simple interaction could be used to support older adults in a home environment.

11 References

- [1] World Health Organization, “Dementia,” 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/dementia>. [Accessed: 23-Apr-2026].
- [2] NHS, “Alzheimer’s disease,” [Online]. Available: <https://www.nhs.uk/conditions/alzheimers-disease/>. [Accessed: 23-Apr-2026].
- [3] Alzheimer’s Society, “Dementia symptoms and signs,” [Online]. Available: <https://www.alzheimers.org.uk/about-dementia/stages-and-symptoms/dementia-symptoms>. [Accessed: 23-Apr-2026].
- [4] S. Palmdorf, C. Sundin, N. Tornkvist and A. Sonde, “Technology-Assisted Home Care for People With Dementia and Their Relatives: A Scoping Review,” *Journal of Medical Internet Research*, vol. 23, no. 1, 2021.
- [5] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, M. Sung, C.-L. Chang and M. Grundmann, “MediaPipe Hands: On-device Real-time Hand Tracking,” *arXiv:2006.10214*, 2020.
- [6] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] scikit-learn, “RandomForestClassifier,” [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>. [Accessed: 23-Apr-2026].

[8] face_recognition documentation, “Face Recognition,” [Online]. Available: <https://face-recognition.readthedocs.io/>. [Accessed: 23-Apr-2026].

[9] OpenCV, “Introduction,” [Online]. Available: <https://docs.opencv.org/4.x/d1/dfb/intro.html>. [Accessed: 23-Apr-2026].

[10] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” [Online]. Available: <https://docs.python.org/3/library/sqlite3.html>. [Accessed: 23-Apr-2026].

[11] SQLite, “Documentation,” [Online]. Available: <https://sqlite.org/docs.html>. [Accessed: 23-Apr-2026].

[12] FastAPI, “WebSockets,” [Online]. Available: <https://fastapi.tiangolo.com/advanced/websockets/>. [Accessed: 23-Apr-2026].

[13] React, “React,” [Online]. Available: <https://react.dev/>. [Accessed: 23-Apr-2026].

[14] pytest documentation, “pytest: helps you write better programs,” [Online]. Available: <https://docs.pytest.org/>. [Accessed: 23-Apr-2026].

[15] Coverage.py documentation, "Coverage.py documentation," [Online]. Available: <https://coverage.readthedocs.io/>. [Accessed: 23-Apr-2026].